

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Отчёт

По лабораторной работе №4

«Анализ трафика компьютерных систем с помощью утилиты Wireshark»

По Компьютерным Сетям

Выполнил:

студент 3 курса

Батманов Даниил Евгеньевич

Группа: Р3307

Приняла:

Авксентьева Елена Юрьевна

г. Санкт-Петербург, 2025

Цель работы

Цель работы – изучить структуру протокольных блоков данных, анализируя реальный трафик на компьютере студента с помощью бесплатно распространяемой утилиты Wireshark.

В процессе выполнения домашнего задания выполняются наблюдения за передаваемым трафиком с компьютера пользователя в Интернет и в обратном направлении. Применение специализированной утилиты Wireshark позволяет наблюдать структуру передаваемых кадров, пакетов и сегментов данных различных сетевых протоколов. При выполнении УИР рекомендуется выполнить анализ последовательности команд и определить назначение служебных данных, используемых для организации обмена данными в протоколах: ARP, DNS, FTP, HTTP, DHCP.

Этап 4.1. Анализ трафика утилиты ping

Необходимо отследить и проанализировать трафик, создаваемый утилитой ping, запустив её следующим образом из командной строки: “ping -l размер_пакета адрес_сайта_по_варианту”. В качестве “размера_пакета” необходимо поочерёдно использовать различные значения от 100 до 10000, самостоятельно выбрав шаг изменения.

Захват из Беспроводная сеть 2

Файл Правка Вид Запуск Захват Анализ Статистика Телефония Беспроводная связь Инструменты Справка

ip.addr == 77.234.196.4

No.	Time	Source	Destination	Protocol	Length	Info
7521	165.178571	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=4440) [Reassembled in #7522]
7522	165.178571	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=12/3072, ttl=51 (request in 7517)
7525	166.168454	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=6c66) [Reassembled in #7527]
7526	166.168454	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=6c66) [Reassembled in #7527]
7527	166.168454	192.168.0.103	77.234.196.4	ICMP	82	Echo (ping) request id=0x0001, seq=13/3328, ttl=128 (reply in 7530)
7528	166.233419	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=4444) [Reassembled in #7530]
7529	166.233419	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=4444) [Reassembled in #7530]
7530	166.233419	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=13/3328, ttl=51 (request in 7527)
7532	167.178689	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=6c67) [Reassembled in #7534]
7533	167.178689	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=6c67) [Reassembled in #7534]
7534	167.178689	192.168.0.103	77.234.196.4	ICMP	82	Echo (ping) request id=0x0001, seq=14/3584, ttl=128 (reply in 7537)
7535	167.219334	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=4446) [Reassembled in #7537]
7536	167.219334	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=4446) [Reassembled in #7537]
7537	167.219334	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=14/3584, ttl=51 (request in 7534)

Frame 7530: 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface \Device\NPF... (0000 1c ce 51 da 10 1e 28 87 ba ae b9 26 08 00 45 00) ::Q::: (8:::8:::E

Ethernet II, Src: TPLink_ae:b9:26 (28:87:ba:ae:b9:26), Dst: AzureWaveTec_da:10:1e (1c:ce:51:da:10:1e)

Internet Protocol Version 4, Src: 77.234.196.4, Dst: 192.168.0.103

Frame (frame), 82 байта

Frame (82 bytes) Reassembled IPv4 (3008 bytes) Пакеты: 7793 · Отображено: 48 (0.6%) Профиль: Default

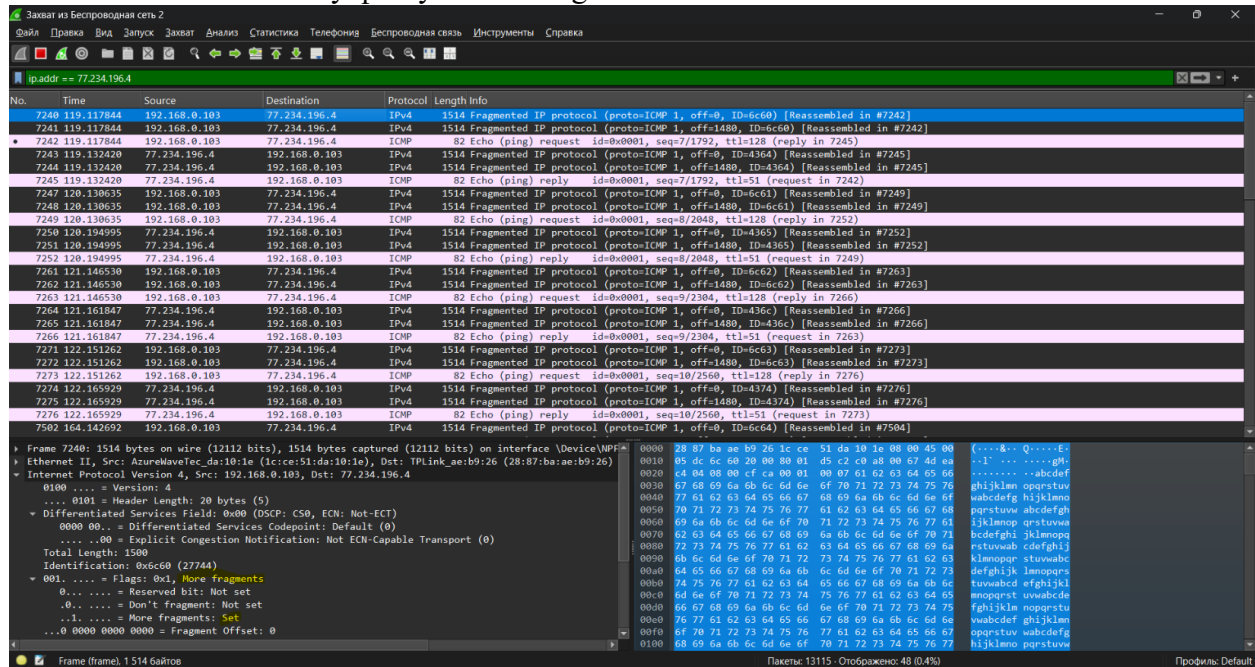
Статистика Ping для 77.234.196.4:
Пакетов: отправлено = 4, получено = 0
(0% потерь)
Приблизительное время приема-передачи в мс:
Минимальное = 14мсек, Максимальное = 64 мсек, Среднее = 26 мсек
PS C:\Users\DanilBatmanov> ping -l 3000 se.ifmo.ru

Обмен пакетами с se.ifmo.ru [77.234.196.4] с 3000 байтами данных:
Ответ от 77.234.196.4: число байт=3000 время=36мс TTL=51
Ответ от 77.234.196.4: число байт=3000 время=15мс TTL=51
Ответ от 77.234.196.4: число байт=3000 время=65мс TTL=51
Ответ от 77.234.196.4: число байт=3000 время=40мс TTL=51

Статистика Ping для 77.234.196.4:
Пакетов: отправлено = 4, получено = 0
(0% потерь)
Приблизительное время приема-передачи в мс:
Минимальное = 15мсек, Максимальное = 65 мсек, Среднее = 39 мсек
PS C:\Users\DanilBatmanov>

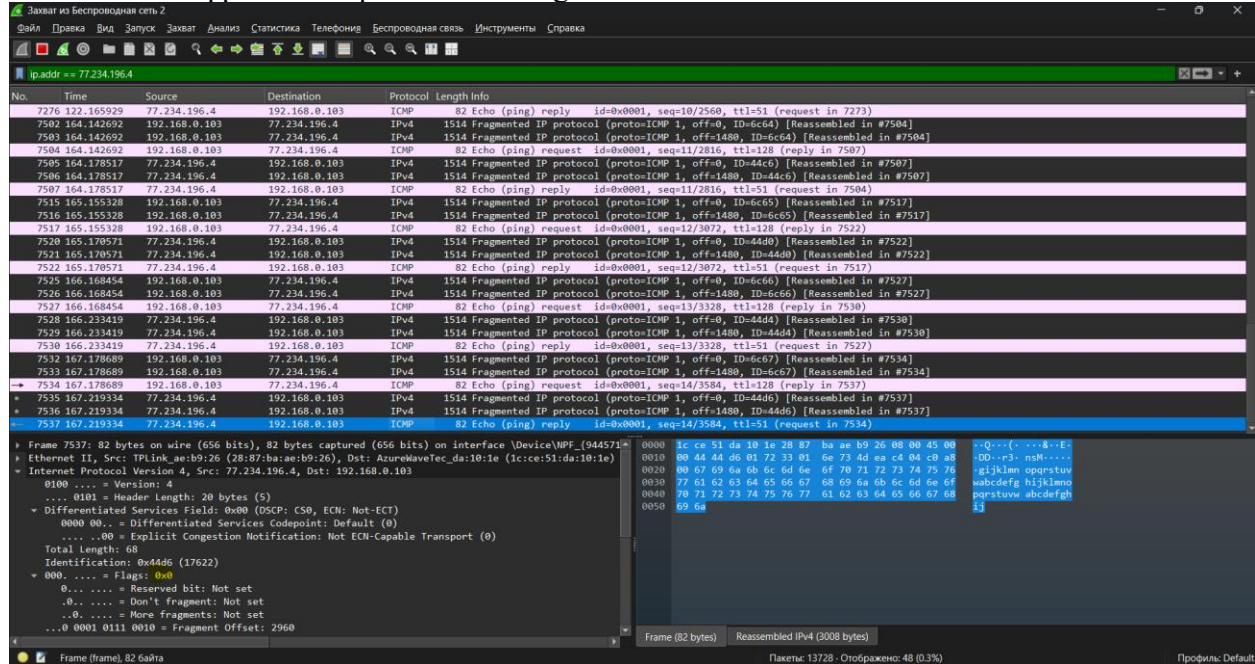
1) Имеет ли место фрагментация исходного пакета, какое поле на это указывает?

Да, имеет, можем это заметить, как минимум по дублированию исходящих сообщений, а также по выставленному флагу «More fragments»



2) Какая информация указывает, является ли фрагмент пакета последним или промежуточным?

У последнего фрагмента флаг «More fragments» выставляется в 0:



У промежуточного – в 1:

The screenshot shows the Wireshark interface. The top pane displays a list of network packets. The middle pane shows the details of the selected packet (No. 7525), which is an Echo (ping) request. The bottom pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
7276	122.145929	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=10/2560, ttl=51 (request in 7273)
7502	164.142692	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=6c64) [Reassembled in #7504]
7503	164.142692	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=6c64) [Reassembled in #7504]
7504	164.142692	192.168.0.103	77.234.196.4	ICMP	82	Echo (ping) request id=0x0001, seq=11/2816, ttl=128 (reply in 7507)
7505	164.178517	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=44c6) [Reassembled in #7507]
7506	164.178517	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=44c6) [Reassembled in #7507]
7507	164.178517	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=11/2816, ttl=51 (request in 7504)
7515	165.155328	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=6c65) [Reassembled in #7517]
7516	165.155328	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=6c65) [Reassembled in #7517]
7517	165.155328	192.168.0.103	77.234.196.4	ICMP	82	Echo (ping) request id=0x0001, seq=12/3072, ttl=128 (reply in 7522)
7520	165.178571	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=44d0) [Reassembled in #7522]
7521	165.178571	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=44d0) [Reassembled in #7522]
7522	165.178571	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=12/3072, ttl=51 (request in 7517)
7525	166.168454	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=6c66) [Reassembled in #7527]
7526	166.168454	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=6c66) [Reassembled in #7527]
7527	166.168454	192.168.0.103	77.234.196.4	ICMP	82	Echo (ping) request id=0x0001, seq=13/3328, ttl=128 (reply in 7530)
7528	166.233419	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=44d4) [Reassembled in #7530]
7529	166.233419	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=44d4) [Reassembled in #7530]
7530	166.233419	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=13/3328, ttl=51 (request in 7527)
7532	167.178689	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=6c67) [Reassembled in #7534]
7533	167.178689	192.168.0.103	77.234.196.4	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=6c67) [Reassembled in #7534]
7534	167.178689	192.168.0.103	77.234.196.4	ICMP	82	Echo (ping) request id=0x0001, seq=14/3584, ttl=128 (reply in 7537)
7535	167.219334	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=44d6) [Reassembled in #7537]
7536	167.219334	77.234.196.4	192.168.0.103	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=44d6) [Reassembled in #7537]
7537	167.219334	77.234.196.4	192.168.0.103	ICMP	82	Echo (ping) reply id=0x0001, seq=14/3584, ttl=51 (request in 7534)

Frame 7525: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface \Device\NPF...
Ethernet II, Src: AzureWaveTec_da:10:1e (1c:ce:51:da:10:1e), Dst: TPLink_ae:b9:26 (28:87:ba:ae:b9:26)
Internet Protocol Version 4, Src: 192.168.0.103, Dst: 77.234.196.4
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
0000 00.. = Differentiated Services Codepoint: Default (0)
.... 00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
Total Length: 1500
Identification: 0x6c66 (27750)
001. = Flags: 0x1, More fragments
0... = Reserved bit: Not set
.0... = Don't fragment: Not set
..1. = More fragments: Set
0 0000 0000 0000 = Fragment Offset: 0

3) Чему равно количество фрагментов при передаче ping-пакетов?

Количество фрагментов зависит от длины сообщения, в данном случае – один фрагмент длиной = 1500 байт, тогда 5000 байт/1500 будет 4 фрагмента (округление вверх)

The screenshot shows the details of Frame 7525. The IP header shows the source as 192.168.0.103 and the destination as 77.234.196.4. The ICMP header shows the type as Echo (ping) request and the sequence number as 14/3584.

Frame 7525: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface \Device\NPF...
Ethernet II, Src: AzureWaveTec_da:10:1e (1c:ce:51:da:10:1e), Dst: TPLink_ae:b9:26 (28:87:ba:ae:b9:26)
Internet Protocol Version 4, Src: 192.168.0.103, Dst: 77.234.196.4
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
0000 00.. = Differentiated Services Codepoint: Default (0)
.... 00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
Total Length: 1500
Identification: 0x6c66 (27750)
001. = Flags: 0x1, More fragments
0... = Reserved bit: Not set
.0... = Don't fragment: Not set
..1. = More fragments: Set
0 0000 0000 0000 = Fragment Offset: 0

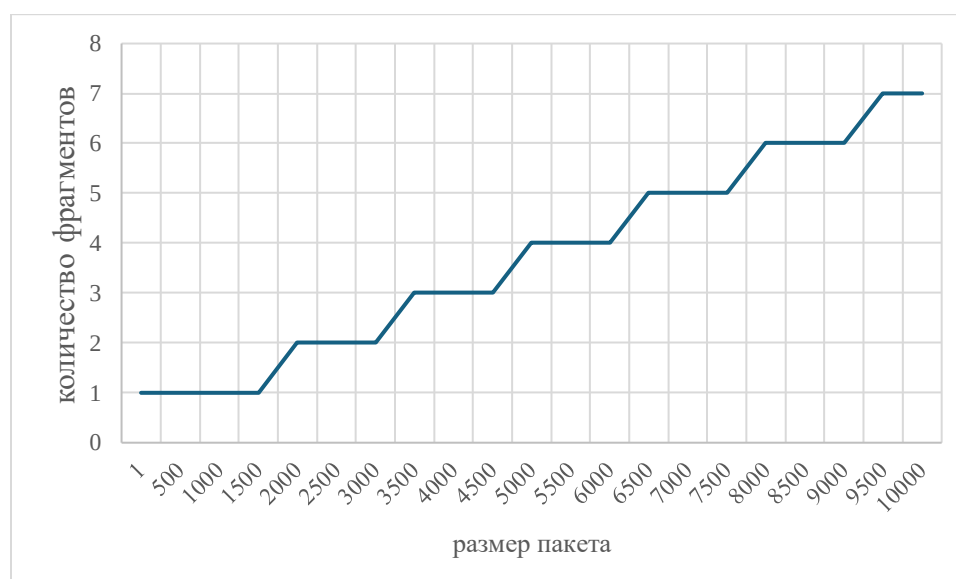
Последний фрагмент полный:

```

▶ Frame 7537: 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface \Device\NPF_{944571...
▶ Ethernet II, Src: TPLink_ae:b9:26 (28:87:ba:ae:b9:26), Dst: AzureWaveTec_da:10:1e (1c:ce:51:da:10:1e)
▼ Internet Protocol Version 4, Src: 77.234.196.4, Dst: 192.168.0.103
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▼ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 68
    Identification: 0x44d6 (17622)
    ▼ 000. .... = Flags: 0x0
        0... .... = Reserved bit: Not set
        .0.. .... = Don't fragment: Not set
        ..0. .... = More fragments: Not set
        ...0 0001 0111 0010 = Fragment Offset: 2960

```

- 4) Построить график, в котором на оси абсцисс находится размер пакета, а по оси ординат – количество фрагментов, на которое был разделён каждый ping-пакет.



- 5) Как изменить поле TTL с помощью утилиты ping?

С помощью флага `-i`, задается в миллисекундах. Например, потеря всех пакетов в следствие `time-out`:

```

PS C:\Users\DaniilBatmanov> ping -l 10000 -i 10 se.ifmo.ru

Обмен пакетами с se.ifmo.ru [77.234.196.4] с 10000 байтами данных:
Ответ от 185.141.124.165: Превышен срок жизни (TTL) при передаче пакета.
Ответ от 185.141.124.165: Превышен срок жизни (TTL) при передаче пакета.
Ответ от 185.141.124.165: Превышен срок жизни (TTL) при передаче пакета.
Ответ от 185.141.124.165: Превышен срок жизни (TTL) при передаче пакета.

Статистика Ping для 77.234.196.4:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потеря)
PS C:\Users\DaniilBatmanov> |

```


6) Что содержится в поле данных ring-пакета?

Мусорные значения:

0000	28 87 ba ae b9 26 1c ce 51 da 10 1e 08 00 45 00	(...&... Q...E...
0010	05 dc 6c 6b 22 e4 0a 01 48 d4 c0 a8 00 67 4d ea	..lk"... H...gM..
0020	c4 04 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f	..bcdefg hijklmno
0030	70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68	pqrstuvwxyz abcdefgh
0040	69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61	ijklmnop qrstuvw
0050	62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71	bcdefghi jklmnopq
0060	72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a	rstuvwab cdefghij
0070	6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63	klmnopqr stuvwabc
0080	64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73	defghijk lmnopqrs
0090	74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c	tuvwabcd efghijkl
00a0	6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65	mnopqrst uvwabcde
00b0	66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75	fghijklm nopqrstu
00c0	76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e	vwabcdef ghijklmn
00d0	6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66 67	opqrstuv wabcdefg
00e0	68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77	hijklmno pqrstuvwxyz
00f0	61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70	abcdefghijklmnopqrstuvwxyz
0100	71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69	qrstuvwxyz abcdefghi
0110	6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62	ijklmnopq rstuvwab
0120	63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72	cdefghij klmnopqr
0130	73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b	stuvwabc defghijk
0140	6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64	lmnopqrs tuvwabcd
0150	65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74	efghijkl mnopqrst
0160	75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d	uvwabcde fghijklm
0170	6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66	nopqrstu vwabcdef
0180	67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76	ghijklmn opqrstuv
0190	77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f	wabcdefg hijklmno
01a0	70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68	pqrstuvwxyz abcdefgh
01b0	69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61	ijklmnop qrstuvw
01c0	62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71	bcdefghi jklmnopq
01d0	72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a	rstuvwab cdefghij
01e0	6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63	klmnopqr stuvwabc
01f0	64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73	defghijk lmnopqrs
0200	74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c	tuvwabcd efghijkl
0210	6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65	mnopqrst uvwabcde
0220	66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75	fghijklm nopqrstu
0230	76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e	vwabcdef ghijklmn
0240	6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66 67	opqrstuv wabcdefg
0250	68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77	hijklmno pqrstuvwxyz
0260	61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70	abcdefghijklmnopqrstuvwxyz
0270	71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69	qrstuvwxyz abcdefghi
0280	6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62	ijklmnopq rstuvwab
0290	63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72	cdefghij klmnopqr
02a0	73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b	stuvwabc defghijk
02b0	6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64	lmnopqrs tuvwabcd

Этап 4.2. Анализ трафика утилиты tracert (traceroute)

Необходимо отследить и проанализировать трафик, создаваемый утилитой tracert (или traceroute в Linux), запустив её следующим образом из командной строки:
“tracert -d адрес_сайта_по_варианту”

The screenshot displays two windows. The top window is Wireshark, showing a packet capture of ICMP Echo (ping) requests and replies. The bottom window is Windows PowerShell, showing the output of the command `tracert -d se.ifmo.ru`.

Wireshark Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
37727	3034.091442	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=53/13568, ttl=12 (no response found!)
37729	3034.097360	194.85.36.66	192.168.0.103	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
37730	3034.098607	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=54/13824, ttl=12 (no response found!)
37731	3034.102596	194.85.36.66	192.168.0.103	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
37737	3035.115919	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=55/14080, ttl=13 (no response found!)
37786	3039.088181	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=56/14336, ttl=13 (no response found!)
37844	3043.102328	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=57/14592, ttl=13 (no response found!)
37892	3047.104808	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=58/14848, ttl=14 (reply in 37893)
37893	3047.117904	77.234.196.4	192.168.0.103	ICMP	106	Echo (ping) reply id=0x0001, seq=58/14848, ttl=51 (request in 37892)
37894	3047.119476	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=59/15104, ttl=14 (reply in 37895)
37895	3047.131877	77.234.196.4	192.168.0.103	ICMP	106	Echo (ping) reply id=0x0001, seq=59/15104, ttl=51 (request in 37894)
37896	3047.133027	192.168.0.103	77.234.196.4	ICMP	106	Echo (ping) request id=0x0001, seq=60/15360, ttl=14 (reply in 37897)
37897	3047.145893	77.234.196.4	192.168.0.103	ICMP	106	Echo (ping) reply id=0x0001, seq=60/15360, ttl=51 (request in 37896)

Windows PowerShell Output:

```
PS C:\Users\DanilBatmanov> tracert -d se.ifmo.ru
Трассировка маршрута к se.ifmo.ru [77.234.196.4]
с максимальным числом прыжков 30:

 1  2 ms    2 ms    2 ms    192.168.0.1
 2  4 ms    4 ms    2 ms    10.82.243.254
 3  *        *        *        Превышен интервал ожидания для запроса.
 4  17 ms   5 ms    3 ms    10.11.13.129
 5  3 ms    3 ms    3 ms    10.11.13.145
 6  3 ms    3 ms    3 ms    10.11.13.130
 7  5 ms    6 ms    3 ms    10.11.13.113
 8  *       63 ms  17 ms   109.239.131.129
 9  10 ms   7 ms    4 ms    91.108.51.3
10  *       *        *        Превышен интервал ожидания для запроса.
11  *       4 ms   4 ms    194.85.42.129
12  *       6 ms   4 ms    194.85.36.66
13  *       *        *        Превышен интервал ожидания для запроса.
14  13 ms  12 ms   13 ms   77.234.196.4

Трассировка завершена.
PS C:\Users\DanilBatmanov>
```

1) Сколько байт содержится в заголовке IP? Сколько байт содержится в поле данных?

В заголовке – 20 байт, а в теле данных – 64.

```

▼ Internet Protocol Version 4, Src: 192.168.0.103, Dst: 77.234.196.4
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▼ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 92
  Identification: 0x6c94 (27796)
  ▶ 000. .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 14
  Protocol: ICMP (1)
  Header Checksum: 0x6d0f [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.0.103
  Destination Address: 77.234.196.4
  [Stream index: 28]

▼ Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0xf7c3 [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 59 (0x003b)
  Sequence Number (LE): 15104 (0x3b00)
  [Response frame: 37895]
  ▶ Data (64 bytes)

```

2) Как и почему изменяется поле TTL в следующих друг за другом ICMP-пакетах tracerf?

37027	2989.248783	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=19/4864	ttl=1	(no response found!)
37028	2989.242772	192.168.0.1	192.168.0.103	ICMP	126 Time-to-live exceeded (Time to live exceeded)		ttl=1	(no response found!)
37029	2989.244260	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=20/5120	ttl=1	(no response found!)
37030	2989.246273	192.168.0.1	192.168.0.103	ICMP	126 Time-to-live exceeded (Time to live exceeded)		ttl=1	(no response found!)
37031	2989.246961	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=21/5376	ttl=1	(no response found!)
37032	2989.247650	192.168.0.1	192.168.0.103	ICMP	126 Time-to-live exceeded (Time to live exceeded)		ttl=1	(no response found!)
37070	2990.261816	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=22/5632	ttl=2	(no response found!)
37071	2990.265805	10.82.243.254	192.168.0.103	ICMP	134 Time-to-live exceeded (Time to live exceeded)		ttl=2	(no response found!)
37072	2990.267132	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=23/5888	ttl=2	(no response found!)
37073	2990.271175	10.82.243.254	192.168.0.103	ICMP	134 Time-to-live exceeded (Time to live exceeded)		ttl=2	(no response found!)
37074	2990.272028	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=24/6144	ttl=2	(no response found!)
37075	2990.274327	10.82.243.254	192.168.0.103	ICMP	134 Time-to-live exceeded (Time to live exceeded)		ttl=2	(no response found!)
37082	2991.285150	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=25/6400	ttl=3	(no response found!)
37134	2995.089677	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=26/6656	ttl=3	(no response found!)
37193	2999.104695	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=27/6912	ttl=3	(no response found!)
37253	3003.103354	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=28/7168	ttl=4	(no response found!)
37254	3003.111043	10.11.13.129	192.168.0.103	ICMP	110 Time-to-live exceeded (Time to live exceeded)		ttl=4	(no response found!)
37255	3003.121209	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=29/7424	ttl=4	(no response found!)
37256	3003.126402	10.11.13.129	192.168.0.103	ICMP	134 Time-to-live exceeded (Time to live exceeded)		ttl=4	(no response found!)
37257	3003.128187	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=30/7680	ttl=4	(no response found!)
37258	3003.131771	10.11.13.129	192.168.0.103	ICMP	134 Time-to-live exceeded (Time to live exceeded)		ttl=4	(no response found!)
37269	3004.140127	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=31/7936	ttl=5	(no response found!)
37270	3004.143449	10.11.13.145	192.168.0.103	ICMP	110 Time-to-live exceeded (Time to live exceeded)		ttl=5	(no response found!)
37271	3004.144219	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=32/8192	ttl=5	(no response found!)
37272	3004.147442	10.11.13.145	192.168.0.103	ICMP	110 Time-to-live exceeded (Time to live exceeded)		ttl=5	(no response found!)
37273	3004.148458	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=33/8448	ttl=5	(no response found!)
37274	3004.152228	10.11.13.145	192.168.0.103	ICMP	110 Time-to-live exceeded (Time to live exceeded)		ttl=5	(no response found!)
37279	3005.154378	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=34/8704	ttl=6	(no response found!)
37280	3005.157711	10.11.13.130	192.168.0.103	ICMP	70 Time-to-live exceeded (Time to live exceeded)		ttl=6	(no response found!)
37281	3005.161340	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=35/8960	ttl=6	(no response found!)
37282	3005.164472	10.11.13.130	192.168.0.103	ICMP	70 Time-to-live exceeded (Time to live exceeded)		ttl=6	(no response found!)
37283	3005.165738	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=36/9216	ttl=6	(no response found!)
37284	3005.167040	10.11.13.130	192.168.0.103	ICMP	70 Time-to-live exceeded (Time to live exceeded)		ttl=6	(no response found!)
37321	3006.177283	192.168.0.103	77.234.196.4	ICMP	106 Echo (ping) request	id=0x0001, seq=37/9472	ttl=7	(no response found!)
37322	3006.183029	10.11.13.113	192.168.0.103	ICMP	110 Time-to-live exceeded (Time to live exceeded)		ttl=7	(no response found!)

Утилита tracerf отправляет первый пакет с TTL равным 1 и увеличивает значение TTL на 1 для каждого последующего отправляемого пока назначение не ответит или пока не будет достигнуто максимальное значение поля TTL. Поскольку каждый маршрутизатор на пути обязан уменьшать значение поля TTL пакета, по крайней мере на 1 перед дальнейшей пересылкой пакета, значение TTL по сути является эффективным счетчиком переходов.

3) Чем отличаются ICMP-пакеты, генерируемые утилитой tracerf, от ICMP пакетов, генерируемых утилитой ping (см. предыдущее задание).

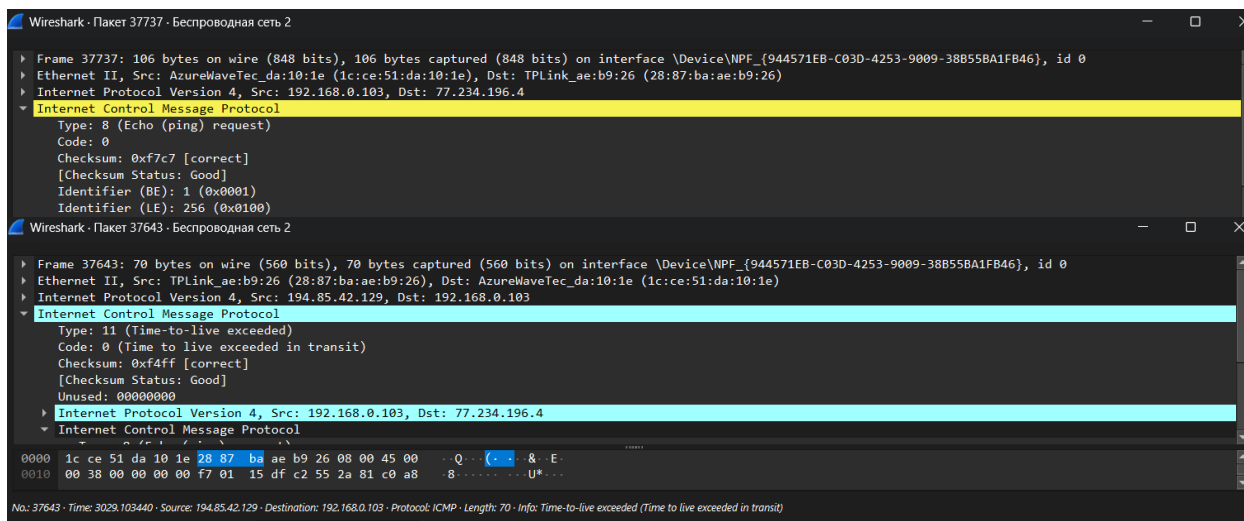
ping-пакеты могут быть различного размера, а также содержат установленный ttl, в то время как пакеты tracerf имеют фиксированный размер, но инкрементирующийся ttl

0000	1c ce 51 da 10 1e 28 87 ba ae b9 26 08 00 45 00	..Q...(&..E..
0010	00 5c a7 e0 00 00 33 01 0c c3 4d ea c4 04 c0 a8	.\...3...M....
0020	00 67 00 00 ff c4 00 01 00 3a 00 00 00 00 00 00	g.....:.....
0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0060	00 00 00 00 00 00 00 00 00 00	

В отличие от пакетов, генерируемых утилитой ping, пакеты, генерируемые утилитой tracerf, в поле данных содержат нули.

- 4) Чем отличаются полученные пакеты «ICMP reply» от «ICMP error» и зачем нужны оба этих типа ответов?

ICMP reply – валидный ответ от хоста, когда сообщение достигло хоста. Имеет тип 0.
ICMP error (вероятно, “ttl exceed in transit”) – время жизни пакета истекло, возвращается, когда ttl=1; имеет тип 11



- 5) Что изменится в работе tracerf, если убрать ключ “-d”? Какой дополнительный трафик при этом будет генерироваться?

Появилась информация об именах хостов:

```

PS C:\Users\DaniilBatmanov> tracert se.ifmo.ru

Трассировка маршрута к se.ifmo.ru [77.234.196.4]
с максимальным числом прыжков 30:

 1    22 ms    4 ms    4 ms  192.168.0.1
 2     4 ms    6 ms    3 ms  10.82.243.254
 3     *      *      *      Превышен интервал ожидания для запроса.
 4    18 ms    4 ms    4 ms  10.11.13.129
 5     6 ms    3 ms    3 ms  10.11.13.145
 6    20 ms    7 ms    2 ms  10.11.13.130
 7    24 ms    7 ms    3 ms  10.11.13.113
 8     6 ms    5 ms    4 ms  109.239.131.129
 9    26 ms    6 ms    4 ms  91.108.51.3
10     4 ms    4 ms    3 ms  2606.ae3.bm18-5-gw.spb.niks.su [185.141.124.165]
11     4 ms    3 ms    4 ms  et014.gw5.kt12.spb.niks.su [194.85.42.129]
12     *      4 ms    6 ms  3557.ae1.kt12-5-gw.spb.niks.su [194.85.36.66]
13     *      *      *      Превышен интервал ожидания для запроса.
14    61 ms   14 ms   12 ms  helios.cs.ifmo.ru [77.234.196.4]

Трассировка завершена.
PS C:\Users\DaniilBatmanov>
PS C:\Users\DaniilBatmanov> tracert -d se.ifmo.ru

Трассировка маршрута к se.ifmo.ru [77.234.196.4]
с максимальным числом прыжков 30:

 1     1 ms    1 ms    2 ms  192.168.0.1
 2     9 ms    4 ms    2 ms  10.82.243.254
 3     *      *      *      Превышен интервал ожидания для запроса.
 4     2 ms    3 ms    2 ms  10.11.13.129
 5     4 ms    4 ms    5 ms  10.11.13.145
 6     8 ms    5 ms    4 ms  10.11.13.130
 7    11 ms    4 ms    4 ms  10.11.13.113
 8    80 ms    *      24 ms  109.239.131.129
 9     6 ms    5 ms    5 ms  91.108.51.3
10     6 ms    7 ms    3 ms  185.141.124.165
11     *      *      6 ms  194.85.42.129
12     5 ms    9 ms    4 ms  194.85.36.66
13     *      *      *      Превышен интервал ожидания для запроса.
14    32 ms   28 ms   13 ms  77.234.196.4

Трассировка завершена.

```

Ключ -d предотвращает попытки команды tracert разрешения IP-адресов промежуточных маршрутизаторов в имена, то есть в выводе будут отсутствовать имена хостов, через которые проходит IP-пакет.

Этап 4.5. Анализ ARP-трафика

Необходимо отследить и проанализировать трафик протокола ARP, сгенерированный в результате выполнения следующих действий:

- очистить ARP-таблицу командой “netsh interface ip delete arpccache” (проверить
- очистилась ли таблица можно с помощью команды “arp -a”, выводящей таблицу на

- экран);
- очистить кэш браузера;
- зайти на Интернет-сайт, заданный по варианту.

```
PS C:\Users\DaniilBatmanov> netsh interface ip delete arpcache
OK.
```

```
PS C:\Users\DaniilBatmanov> arp -a
```

```
Интерфейс: 192.168.0.103 --- 0хa
```

адрес в Интернете	Физический адрес	Тип
192.168.0.1	28-87-ba-ae-b9-26	динамический
224.0.0.22	01-00-5e-00-00-16	статический

```
PS C:\Users\DaniilBatmanov> arp -a
```

```
Интерфейс: 192.168.0.103 --- 0хa
```

адрес в Интернете	Физический адрес	Тип
192.168.0.1	28-87-ba-ae-b9-26	динамический
224.0.0.22	01-00-5e-00-00-16	статический
224.0.0.251	01-00-5e-00-00-fb	статический

```
PS C:\Users\DaniilBatmanov> |
```

- 1) Какие MAC-адреса присутствуют в захваченных пакетах ARP-протокола? Что означают эти адреса? Какие устройства они идентифицируют?

№	Time	Source	Destination	Protocol	Length	Info
87227	5317.184118	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.102? Tell 192.168.0.1
87228	5318.216278	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.104? Tell 192.168.0.1
87229	5318.216329	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.105? Tell 192.168.0.1
87230	5318.216344	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.100? Tell 192.168.0.1
87231	5319.235924	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.100? Tell 192.168.0.1
87232	5320.259124	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.104? Tell 192.168.0.1
87232	5320.259124	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.104? Tell 192.168.0.1
87233	5320.259159	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.100? Tell 192.168.0.1
87234	5320.259167	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.100? Tell 192.168.0.1
87235	5321.187198	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.104? Tell 192.168.0.1
87236	5321.187234	TPLink_ae:b9:26	Broadcast	ARP	42	Who has 192.168.0.100? Tell 192.168.0.1

```

> Frame 87273: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{944571E
> Ethernet II, Src: AzureWaveTec_da:10:1e (1c:ce:51:da:10:1e), Dst: TPLink_ae:b9:26 (28:87:ba:ae:b9:26)
▼ Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: AzureWaveTec_da:10:1e (1c:ce:51:da:10:1e)
  Sender IP address: 192.168.0.103
  Target MAC address: TPLink_ae:b9:26 (28:87:ba:ae:b9:26)
  Target IP address: 192.168.0.1

```

```

▶ Frame 87260: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{944571E
▶ Ethernet II, Src: TPLink_ae:b9:26 (28:87:ba:ae:b9:26), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
▼ Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: TPLink_ae:b9:26 (28:87:ba:ae:b9:26)
  Sender IP address: 192.168.0.1
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 192.168.0.107

```

28:87:ba:ae:b9:26 - адрес отправителя (маршрутизатор) (ip = 192.168.0.1)

1c:ce:51:da:10:1e - адрес устройства получателя (компьютер, с которого производится запрос на сайт) (ip = 192.168.0.103)

00:00:00:00:00:00 - broadcast-адрес

- 2) Какие MAC-адреса присутствуют в захваченных HTTP-пакетах и что означают эти адреса? Что означают эти адреса? Какие устройства они идентифицируют?

В HTTP-пакетах присутствуют те же самые MAC-адреса, что и в ARP запросе. Они используются для идентификации отправителя и получателя HTTP-пакета.

- 3) Для чего ARP-запрос содержит IP-адрес источника?

Чтобы узел-получатель мог добавить информацию об узле-отправителе в свою ARP-таблицу.

Этап 4.4. Анализ DNS-трафика

Необходимо отследить и проанализировать трафик протокола DNS, сгенерированный в результате выполнения следующих действий:

- настроить Wireshark-фильтр: “ip.addr == ваш_IP_адрес”;
- очистить кэш DNS с помощью команды ipconfig в командной строке: ipconfig /flushdns
- очистить кэш браузера;
- зайти на Интернет-сайт, заданный по варианту.

```
PS C:\Users\DaniilBatmanov> ipconfig /flushdns
```

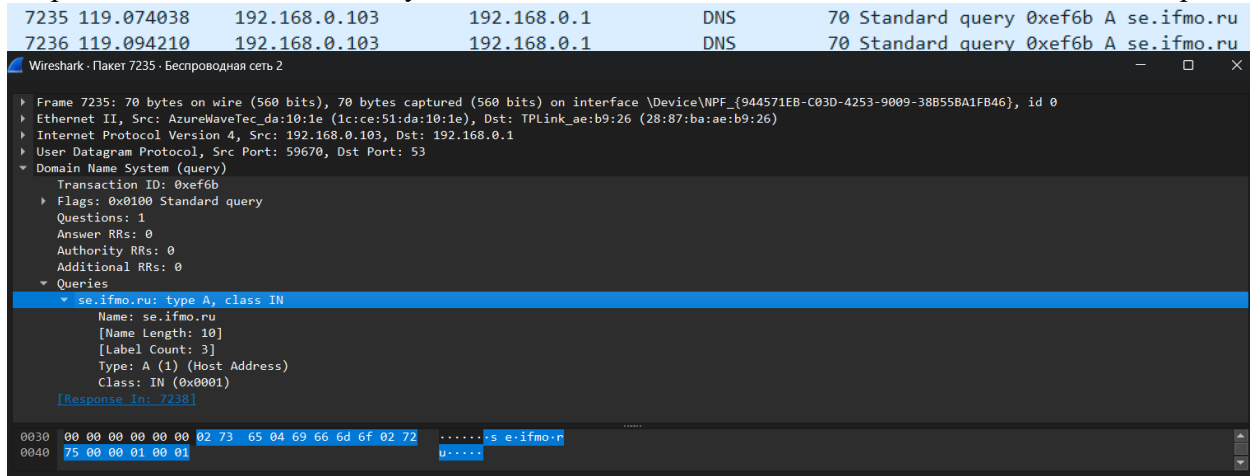
Настройка протокола IP для Windows

Кэш сопоставителя DNS успешно очищен.

```
PS C:\Users\DaniilBatmanov> |
```

- 1) Почему адрес, на который отправлен DNS-запрос, не совпадает с адресом посещаемого сайта?

Так как только что был очищен кэш, необходимо получить с DNS-сервера адрес запрашиваемого сайта. Там будет найдено совпадение доменного имени и сетевого адреса.



- 2) Какие бывают типы DNS-запросов?

Прямой: преобразование домена в IP-адрес.

Обратный: преобразование IP-адреса в домен.

Рекурсивный: выполняется DNS-сервером, пока не будет найден домен или не будет получен ответ, что домен не существует. Рекурсия выполняется сервером.

Итеративный: то же самое, что рекурсивный, но также допускается выполнение поиска клиентом.

- 3) В какой ситуации нужно выполнять независимые DNS-запросы для получения содержащихся на сайте изображений?

В случае, если изображения находятся на другом хосте.

Вывод

Была изучена структура протокольных блоков данных через анализ реального трафика на компьютере студента с помощью бесплатно распространяемой утилиты Wireshark.

Были рассмотрены такие протоколы, как DNS и ARP.

Были использованы такие утилиты, как ping, tracert для пересылки пакетов каких-либо непустых для проверки соединения.